

Assignment 2: Coding Basics

Meg O'Brien

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1. Generating a sequence of numbers from one to 55, increasing by fives.
```

```
sequence1 <- seq(1, 55, 5)
```

```
#2. Compute the mean and median of sequence1.
```

```
mean(sequence1) #The mean is 26.
```

```
## [1] 26
```

```
median(sequence1) #The median is 26.
```

```
## [1] 26
```

```
#3. Determine whether the mean is greater than the median.
```

```
mean(sequence1) > median(sequence1) #R confirms the mean is not greater than the median.
```

```
## [1] FALSE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.

8. Label the columns of your data frame with informative titles.

```
students <- c("Aubrey", "Jacqui", "Sy", "Nina") #Character vector
testscores <- c(92, 88, 81, 99) #Numeric vector
scholarship <- c(TRUE, TRUE, FALSE, FALSE) #Logical vector
student_info <- data.frame(students, testscores, scholarship)
colnames(student_info) <- c("Student_Name", "Test_Score", "Scholarship_Status")
```

9. QUESTION: How is this data frame different from a matrix?

Answer: This data frame is different from a matrix because the columns are able to hold different data types (character, numeric, and logical), where as a matrix can only have one data type.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word "Pass"; otherwise print the word "Fail".
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

#10. Create a function using if...else

```
function1 <- function(x) {
  if (x > 50) {
    print("pass")
  } else {
    print("fail")
  }
}
```

#11. Create a function using ifelse()

```
function2 <- function(x) {
  print(ifelse(x > 50, "pass", "fail"))
}
```

#12a. Run the first function with the value 52.5

```
function1(52.5) #pass
```

```
## [1] "pass"
```

#12b. Run the second function with the value 52.5

```
function2(52.5) #pass
```

```
## [1] "pass"
```

#13a. Run the first function with the vector of test scores

```
#function1(testscores)
```

#13b. Run the second function with the vector of test scores

```
function2(testscores) #"pass" "pass" "pass" "pass"
```

```
## [1] "pass" "pass" "pass" "pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: Only the ‘ifelse’ option worked when running the function with the vector of test scores. This is because the ‘if’ only works with a single value, and this is a vector with multiple values. The ‘ifelse’ is vectorized, meaning that it can be applied to an entire vector. Therefore, the ‘ifelse’ worked with multiple values when running the function with the vector.